

Challenge 2

Overview

In their never-ending quest to innovate, Quinnipiac was very impressed with how well their initiative went to get their eBay-inspired qBay platform out the door, owing much of their success to your proof-of-concept work. The school realized that they could create a new initiative where other companies could request students to build out a prototype for a new idea of theirs, which give the students valuable experience and potential exposure for a future hiring.

Microsoft, the current owner of GitHub, is particularly interested in using this opportunity to come up with a new tool for Git and GitHub. Quinnipiac has enlisted the students involved with the original qBay pilot to once again use their skills to build out a prototype for Microsoft. In return, this is direct exposure to company leaders at Microsoft, and other FAANG company recruiters are keeping a close eye – this could very well be an instant job opportunity with a guaranteed six-figure payout if all goes well!

Assignment

You will work in your assigned team (posted on Blackboard).

Microsoft has specifically requested a GUI (**G**raphic **U**ser **I**nterface) application that will allow a user to easily kick off the process of converting an existing project on their computer into a Git repo, and pushing the repo up to GitHub. This means that your project must be capable of doing the following:

- Turn a project on a user's computer into a Git repo
- Add a Git Ignore file to the project with some common files/patterns to ignore
- Add a Read Me file that contains the name of the project (as a markdown header)
- Create an initial commit in the Git repo
- Create a GitHub repo mirroring the Git repo on the developer's computer
 - The GitHub repo's name, description and visibility (private or public) should be able to be set by the user
- The Git repo's remote is set to the GitHub repo as "origin".
- The initial commit is pushed to the GitHub repo.
- The user is given a url to their newly created GitHub repo.

As a team, you will come up with a tool and build out a prototype program (proof of concept) that can achieve what Microsoft is asking for. Since the application is just a prototype, you do not need to worry about error handling beyond a simple "something went wrong" message if all of the steps were not carried out correctly. The design of the GUI doesn't matter much – just get something out the door! Microsoft has requested that the GUI does include a custom logo that represents both Quinnipiac and Microsoft, as well a disclaimer that the application is a prototype and not yet meant for commercial use.

Resources

In order to assist you all, Microsoft has also offered up two Java packages that can be added to your project – **GitSubprocessClient** and **GitHubApiClient**.

The **GitSubprocessClient** package provides a way to run Git commands from a Java program (as opposed to your terminal).

The **GitHubApiClient** packages essentially contains a wrapper class that allows for easy sending of API calls to GitHub. You can think of the GitHub API as a way for you to both tell GitHub to do something or retrieve information from GitHub, and the **GitHubApiClient** enables this communication to be done from a Java program.

The packages (.jar files) can be downloaded from their repos under the **Releases** section (make sure to grab the most up to date version). The repo for **GitSubprocessClient** can be found [here](#). The repo for **GitHubApiClient** can be found [here](#). Instructions for how to add an external Java package to a project can be found in the **Adding_External_Libraries_to_a_Java_Project.pdf** document. Note that these JAR library files should NOT be pushed up to your GitHub repo, and each team member must download and add these packages to their project locally.

Additional Requirements

Each team must use GitHub to host your project, and also use it as a version control and collaboration tool.

All team members are expected to put in equal work for this project. You will be surprised how quickly an application can be built up with multiple developers working asynchronously through the use of collaborative tools like Git and GitHub. Any team member that is not pulling their weight or not communicating with the rest of the team in a reasonably timely manner will receive a zero on the assignment. Teams are encouraged to reach out to me ASAP if there is a team member not doing their fair share of the work so that I can deal with it appropriately.

A website with a project description and detailed setup/usage instructions for your application is also required from the user's perspective, and should be made in the same repo as the application. Hint: create a **docs** folder in the repo, place your site code in there, and target your site to use it in the GitHub Pages settings. Remember to list requirements in order for someone else to be able to run the application such as Java version minimum, external libraries used (and where to find them and the version number), etc.

A team's repo must include a Read Me file in their root project folder that summarizes their application prototype and how to run it from a developer's perspective. It should contain at least two sections – a brief summary of the application, and how to run the application as a developer – don't forget to list requirements such as Java version minimum, external libraries used (and where to find them and the version number, etc.).

In the repo's **About** section, the repo must have a set description, and the **Website** field must be set to url of the website created for this application.

The repo should also include a Git ignore file that will properly ignore all team member IDE specific files and other undesirable files. There should be no “unnecessary” files in your repo (defined by the types of files we went over in class that should be omitted from version control).

Repo Setup

One team member on each assigned team must set up the GitHub repo to be used for the team, and then add their other team members to it as collaborators. They must also set the repo up with branch protection rules that disallow pushing directly to the **master** branch, and for pull requests to require approvals before they are allowed to be merged in. The **Branch_Protection_Rules.pdf** document contains information on how to do this. Note that an initial commit should be created directly to the **master** branch by the repo owner to start the repo off prior to setting up these branch protection rules.

GitHub API Key

It should be noted that in order to use the provided **GitHubApiClient** package, a developer will need to generate a personal GitHub API token (the repo’s documentation goes over how to do that). Since this token is essentially is a password for your account, it is sensitive information and must NOT be committed to Git. If I find an **active** token committed and on GitHub, the team will receive a zero. If you do run into a situation where a token has made its way up to GitHub, you will have to delete the token from your account and generate a new one in order to not receive this penalty – this is the only way to rectify the security breach, since once a password is in GitHub, it’s compromised.

Advice

Each team should identify their own workflow pattern that works best for them. It’s key to identify which tasks need to be taken care of first, and those that can be done in parallel. Often times, a good strategy is to have several tasks going in parallel, and then have the most experienced engineer of a team to take on the task of “connecting everything together” if there is a need for that.

Additionally, the team as a whole should spend some time learning how to use the two provided Java packages in order to understand their capabilities and how they line up with the various required program features.

Submission

When your project is complete, one member per team must submit a link to the project’s GitHub repository webpage, in addition to a high-level breakdown of which team members worked on which areas of the application.

Grading Rubric

In order to get an A on this assignment (20/20):

Note: half of this assignment is a team grade and half is an individual grade.

Team Grade: 10pts

As a team, you will be graded on your application prototype meeting all requirements, as well as the work output meeting quality standard. You will also be graded on your codebase's health, which essentially is how organized and well-maintained your repo is. If things are a mess, or I have trouble following the Git History, you will lose points. This application should be consistent, as if it were all made by the same developer. There should be no environment specific files in the repo (e.g. everything that should be git ignored based on what we covered in class is properly ignored from the repo). All branches that have already been merged into the **master** branch should be deleted from the GitHub repo. The repo's Read Me file and website must be neat, well-organized, and provide relevant documentation for the project based (remember: the Read Me is for other developers, the website is for users of the application).

Individual Grade: 10pts

As an individual, you will be graded on how well you worked as a team on this project, and that you contributed meaningful changes to the project in order to reach its goal.